SKYWARD / TECHNICAL DOCUMENTATION

FLIGHT BOOKING SYSTEM

Technical Study Guide & Architecture Documentation

Java 17+ | Swing | JDBC | MySQL 8+

Layered MVC-style design: UI -> service -> DAO -> database

Team presentation guide with random-line rewrite practice

OBSIDIAN #1E2229 TEAL #0F766E PROJECT EDITION | 02 NOV 2026

CONTENTS | Guide at a glance

Architecture, business rules, database relationships, source file explanations, and team practice.

SECTION	MODULE
1.0	Executive summary and stack
2.0	Architecture and application flow
3.0	Security and authentication
4.0	Database and route management
5.0	Interactive seats and cabin pricing
6.0	Multi-passenger booking
7.0	Dashboard and boarding passes
8.0	Cancellation and refund engine
9.0	Database ER schema
10.0	Every Java program: file-by-file map
11.0	Random-line rewrite practice
12.0	Presentation checklist and glossary

HOW TO USE THIS GUIDE

Start with the architecture flow. Assign each teammate one source file from Section 10. Use Section 11 for timed missing-line exercises. The checked-in source code is the final authority if behavior changes.

Scope: this document describes the Java Swing application in the repository root. The separate Spring Boot + React project under webapp is not covered here.

1.0 | Executive summary & technology stack

Skyward is a desktop flight booking system for passengers and administrators. It searches flights, assigns seats, stores traveler and ticket records, and simulates checkout and refunds.

LAYER	PROJECT CHOICE	RESPONSIBILITY
Language/runtime	Java 17+	Main method starts Swing AppFrame.
Desktop UI	Java Swing	JFrame, JPanel, JDialog, JTable, JComboBox, JSpinner.
Persistence	JDBC + MySQL Connector/J	PreparedStatement queries and explicit transactions.
Database	MySQL 8+	Schema in schema.sql; SQLite is not configured in this desktop project.
Password hashing	PBKDF2-HMAC-SHA256	310,000 iterations and random salt; legacy BCrypt accepted and upgraded.
Architecture	Layered MVC-style	UI -> service -> DAO -> relational tables.

The project has no dedicated controller package. Swing listeners connect the view to service methods; services apply business rules; DAOs handle SQL and result mapping.

ARCHITECTURE IN ONE LINE

Passenger/admin action -> Swing view -> service validation -> DAO transaction -> MySQL -> model/result -> refreshed Swing view.

Core outcomes

- Search by origin, destination, departure date and maximum Economy fare.
- Book up to eight passengers with different seats and cabin classes.
- Keep bookings and boarding passes attached to the account holder.
- Require administrator approval for passenger cancellation requests.

2.0 | Architecture & application flow

Event-driven Swing views call services and DAOs. Database work stays out of rendering code as much as the current layered design allows.

STEP	CLASSES	ROLE
1. UI	LoginPanel, FlightSearchPanel, AdminPanel	Collect input and display state.
2. Service	AuthService, BookingService	Validate credentials, passengers, seats and allowed state changes.
3. DAO	AccountDao, FlightDao, BookingDao	Execute parameterized SQL and map ResultSet rows.
4. Config	DatabaseConnection	Read JDBC URL, username and password from environment.
5. Database	MySQL schema	Enforce keys, constraints, and persistence.

Flight search sequence

```
FlightSearchPanel search action
  -> FlightDao.search(from, to, date, maxEconomyFare)
  -> PreparedStatement with only the active filters
  -> ResultSet mapped into List<Flight>
  -> JTable model refreshed
```

Seat reservation sequence

```
BookingDialog -> BookingService.validatePassengers
  -> SeatSelectionDialog (initial load + live refresh)
  -> BookingService.create -> BookingDao.create transaction
       lock flight row -> validate every requested seat
       insert booking, passengers, payment, tickets
       commit; on error roll back
```

WHY CHECK TWICE?

The visual map can become stale. It refreshes for convenience, but the final DAO check under a flight row lock is what prevents duplicate assignment.

3.0 | Security & authentication architecture

AuthService and AccountDao authenticate the identity. Admin role is determined from the admins table; the mandatory passkey is an additional check.

3.1 Email and administrator sign-in

```
Passenger: email + password -> AccountDao.authenticate -> PasswordUtil.verify
          -> Account -> AppFrame routes to FlightSearchPanel

Administrator: username + password + masked passkey
          -> AuthService checks password and passkey 20080301
          -> Account -> AppFrame routes to AdminPanel
```

CONTROL	IMPLEMENTATION
Login identifier	Passenger email; administrator username.
Password persistence	PBKDF2 hash with random 16-byte salt and 310,000 rounds; no plaintext stored.
Legacy upgrade	BCrypt hashes are recognized and replaced with PBKDF2 after valid login.
Admin setup	First-admin dialog appears only when admins table is empty; service validates passkey before insert.
Admin every login	Masked JPasswordField prompt; AuthService performs constant-time passkey comparison.
SQL safety	PreparedStatement placeholders bind values instead of concatenating SQL.
Database secret	FLIGHT_DB_PASSWORD environment variable; application account should not be MySQL root.

CLASSROOM PASSKEY

AuthService.java currently contains ADMIN_PASSKEY = "20080301". This requested demo key is visible in source; a deployed application should keep a rotated secret outside version control.

3.2 Password reset

ForgotPasswordDialog accepts passenger email and New Password / Confirm New Password. AuthService validates length and equality; AccountDao stores a new PBKDF2 hash. This desktop flow is a direct local reset form and does not generate a temporary password or send a recovery email.

4.0 | Database & route management

The root desktop schema is MySQL 8. The domestic seed produces 208 upcoming departures: 26 city pairs, both directions, four departure times per direction. A separate sample seed adds 40 records.

4.1 Dataset and budget filter

The seed uses realistic route fares, with many Economy fares below or near ₹4,000. FlightSearchPanel offers a maximum Economy fare field. FlightDao appends the inclusive predicate economy_ticket_price <= ? only when enabled. Past flights and fully occupied flights are excluded.

FILTER	QUERY BEHAVIOR
From / To	Case-insensitive contains match on departure_city and destination_city.
Departure date	Half-open time range: selected date at 00:00 inclusive to next date at 00:00 exclusive.
Max Economy fare	Inclusive cap: economy_ticket_price <= entered maximum.
Departure	Only departure_time >= NOW() is returned.
Seat availability	Active issued tickets counted by cabin; flights with no seats left are removed.

4.2 Obsidian & Pearl UI tokens

TOKEN	VALUE	USAGE
Obsidian	#1E2229	Dark panels and prominent contrast areas.
Teal	#0F766E	Primary actions, focus rings, selected brand accents.
Pearl	#F8F9FA desktop	Main canvas (Ui.java); nearby spec pearl #F8FAFC.
Ink	#111827	Heading and label text.
Muted	#6B7280	Hints and secondary labels.
Input border	#E5E7EB	Field and card outlines.
Gold	#F59E0B	Selected seat highlight.

BUDGET RULE

A fare exactly ₹4,000 passes a ₹4,000 cap. The filter checks Economy base fare even if a traveler subsequently chooses Business Class.

5.0 | Interactive seats & tiered cabin layout

The seat grid has six columns. It leaves an aisle gap between C and D and colors the front Business seats separately.

COLUMN	POSITION	COLUMN	POSITION
A	Window	D	Aisle
B	Middle	E	Middle
C	Aisle	F	Window

Cabin ordinal formula

SEAT MAPPING

ordinal = (row - 1) x 6 + columnIndex + 1, where A=0 and F=5. If ordinal <= businessCapacity, it is Business; later ordinals are Economy.

STATE / CABIN	SHARE / PRICE	VISUAL
Business	About one third; 2.5x-3.0x Economy. Seed uses 2.75x.	Purple, front rows.
Economy	About two thirds; route base fare.	Teal.
Occupied	Active, issued tickets for this flight.	Gray and disabled.
Selected	Seat picked by current user.	Gold highlight.

Live occupancy and concurrency

SeatSelectionDialog loads BookingDao.occupiedSeats(flightId), then polls every 2,500 ms using a Swing Timer and SwingWorker. The DAO matches the exact flight and active booking/ticket status. At final reservation, BookingDao.create locks that flight row with SELECT ... FOR UPDATE, checks every seat again, and commits all writes together. A conflict causes rollback and the user is asked to select another seat.

FRESH DISPLAY VERSUS CORRECTNESS

Polling makes open maps converge quickly. The transaction and lock enforce correctness at checkout; the occupied color alone is never trusted as the final authority.

6.0 | B2C multi-passenger booking workflow

One account holder manages one booking containing multiple travelers and seats. The current UI supports one through eight passenger rows.

STEP	ACTION	WHAT HAPPENS
1	Add traveler	Full name, age, gender, passport number (optional).
2	Validate rows	BookingService enforces 1-8 rows, age 0-120 and allowed gender values.
3	Select seats	One unique seat per passenger; map determines cabin class.
4	Reserve	DAO revalidates under flight lock; writes booking and passenger/ticket data.
5	Demo payment	Simulated method confirms the booking; no bank/card details collected.

Validation rules

- Passenger name required; maximum 120 characters.
- Age range is 0-120; gender is Male, Female, or Other.
- Passport is optional, maximum 32 characters.
- Seat code is row + A-F; no duplicate seats within the same booking.
- Cabin class must match the seat's ordinal and configured flight capacity.
- Booking, passengers, payment row, tickets, and seat inventory are transactionally consistent.

```
Booking total = sum(each passenger's class fare)
DB transaction: setAutoCommit(false) -> write rows -> commit
On exception: rollback all changes
```

Models such as PassengerInput, Flight and BookingSummary are ordinary Java data objects populated by DAOs. They are not ORM entities.

7.0 | Dashboard & digital boarding passes

The passenger dashboard lives in FlightSearchPanel and modal dialogs. Boarding passes are reloaded by account ID and shown per issued ticket.

PASS FIELD	SOURCE
Passenger name, age, gender, passport	passengers row associated with ticket.
Airline, flight number, route	ticket -> flight -> airline.
Departure and arrival	flight schedule.
Seat and cabin	ticket.seat_number and ticket.cabin_class.
Ticket fare and number	ticket_price and unique ticket_number.
Booking reference	booking_id displayed as SB-{bookingId}.

BoardingPassDialog shows Your Boarding Pass with passenger, schedule, destination, seat, cabin, ticket price, booking reference, and ticket number. BookingDao filters passes to the signed-in account and only confirmed bookings with issued tickets. A pass view offers Request cancellation; the booking remains active until admin approval.

ACCOUNT OWNER AND PASSENGER

The account holder owns the booking. Each traveler has their own passenger row and ticket, so one multi-person booking can produce several boarding passes.

8.0 | Time-based cancellation & refund engine

The passenger requests cancellation; an administrator approves or declines. An approval records the refund summary, cancels tickets, and releases the reserved seats.

TIME BEFORE DEPARTURE	RATE	RULE
48 hours or more	100%	Full refund.
12 hours to under 48 hours	50%	Partial refund.
2 hours to under 12 hours	30%	Partial refund.
Under 2 hours	20%	Last-window refund.

EXACT FORMULA

secondsBeforeDeparture = departureTime - cancellationTime
refundAmount = round(originalPaidAmount x refundRate / 100, 2)

The implementation uses Duration/seconds and BigDecimal rounded to two decimals (HALF_UP). An unpaid pending booking has no collected payment; its refundable amount is zero.

Approval state change

```
Request: cancellation_requests = PENDING; seat stays active.
Approve: calculate tier -> save booking_cancellations summary
         -> bookings.status = CANCELLED
         -> tickets.status = CANCELLED
         -> release Economy/Business inventory
         -> update payments.refund_amount/status/date
         -> mark request APPROVED, reviewer and time
Decline: mark request REJECTED; booking and seats stay active.
```

RefundSummary is a Java object, not a JSON payload. Its dialog displays bookingId, flightNumber, route, departureTime, cancelledAt, secondsBeforeDeparture, originalAmount, refundPercentage, refundedAmount, and reason.

SIMULATED ACCOUNTING

The app stores refund values and payment status locally. It does not contact a payment provider or transfer funds.

9.0 | Full database entity relationship schema

The relationships below reflect schema.sql and its cancellation migration.

TABLE	KEYS	IMPORTANT DATA / RELATIONSHIP
users	PK user_id; unique email	Profile and password hash. One user -> many bookings and cancellation requests.
admins	PK admin_id; unique username	Admin identity; may review many cancellation requests.
airlines	PK airline_id; unique airline_name	One airline -> many flights.
flights	PK flight_id; FK airline_id	Route, schedule, Economy/Business capacities and fares.
bookings	PK booking_id; FK user_id, flight_id	Status, total. Belongs to one user and one flight.
passengers	PK passenger_id; FK booking_id	Name, age, gender, passport. One booking -> many passengers.
payments	PK payment_id; unique FK booking_id	Amount, method/status, refund value/date; one payment per booking.
tickets	PK ticket_id; FK booking_id, flight_id, passenger_id	Unique ticket number; seat, cabin, price, status.
cancellation_requests	PK request_id; FK booking, user, reviewed_by	Request state, response and review time; admin ref nullable.
booking_cancellations	PK cancellation_id; unique FK booking_id	Route snapshot, times, seconds, original/refund values, tier reason.

Cardinality map

```
users       1 ---- * bookings       * ---- 1 flights       * ---- 1 airlines
bookings    1 ---- * passengers
bookings    1 ---- 0..1 payments
passengers  1 ---- 0..1 tickets       tickets * ---- 1 flights
bookings    1 ---- * cancellation_requests * ---- 1 users
admins      1 ---- * cancellation_requests (reviewed_by nullable)
bookings    1 ---- 0..1 booking_cancellations (booking_id UNIQUE)
```

The tickets table has an index over flight, ticket status and seat. The final active-seat uniqueness rule is checked transactionally in DAO logic under the flight lock.

10.0 | Source code map: every desktop Java program

Paths are relative to src/main/java/com/flightbooking. Use the explanation column as a short oral-exam answer.

PROGRAM / FILE	EXPLAIN IT THIS WAY
FlightBookingApp.java	Entry point; creates Swing UI on the event-dispatch thread.
ui/AppFrame.java	Owns DAO/services; routes admin to AdminPanel and passenger to FlightSearchPanel.
ui/Ui.java	Shared palette, fonts, controls, table styling and dialogs.
ui/LoginPanel.java	Role selection; password and admin passkey prompts; sign-in routing.
ui/RegisterDialog.java	Passenger form; sends values to AuthService.
ui/CreateAdminDialog.java	First admin creation form; passkey is required.
ui/ForgotPasswordDialog.java	Passenger email and matching new/confirmation passwords.
ui/FlightSearchPanel.java	Search filters, flight JTable, bookings and boarding-pass entry points.
ui/CalendarDialog.java	Calendar date picker; disallows past dates.
ui/BookingDialog.java	Structured passenger rows; opens seat map and checkout.
ui/SeatSelectionDialog.java	A-F cabin map, occupied states and 2.5-second refresh timer.
ui/BoardingPassDialog.java	Displays saved ticket details; sends cancellation request.
ui/RefundSummaryDialog.java	Shows route, time delta, original amount, tier, reason, refund.
ui/AdminPanel.java	Flight CRUD, booking list, user search, cancellation approval.
service/AuthService.java	Auth and input rules; passkey checks; delegates persistence.
service/BookingService.java	Validates passengers, seat choices, and accepted payment method.
dao/AccountDao.java	Account/admin authentication, registration, search, and password reset.
dao/FlightDao.java	Route/fare search, flight result mapping, admin flight writes.
dao/BookingDao.java	Transactions for seats, payment, tickets, cancellations and refunds.
config/DatabaseConnection.java	Loads JDBC URL, user and password from environment.
security/PasswordUtil.java	PBKDF2 hash/verify, BCrypt compatibility, hash upgrade.
model/Account.java	Signed-in account identity and admin role.
model/Airline.java	Airline identity/name for flights.
model/Flight.java	Route, schedule, class capacities and prices.
model/PassengerInput.java	Passenger form data plus seat and cabin.
model/BookingSummary.java	Booking table row and status summary.
model/TicketInfo.java	Ticket confirmation data.
model/BoardingPass.java	Passenger/flight/seat/ticket details displayed in pass modal.
model/RefundSummary.java	Refund timestamp, seconds, amount, percentage and reason.
model/CancellationRequest.java	Admin cancellation review queue row.
model/UserInfo.java	Passenger search/profile summary.

EXPLAIN ANY PROGRAM IN FOUR PARTS

(1) What responsibility does the class own? (2) Which UI event or service calls it? (3) What data enters and leaves? (4) Which invariant or user result does it protect?

11.0 | Random-line rewrite practice

Tutor mode: cover the right column. Give each teammate 45 seconds to rewrite a missing line and explain why it matters.

MISSING-LINE PURPOSE	REWRITE	WHY IT MATTERS
Bind SQL input	p.setString(1, value);	Prevents input from becoming SQL syntax.
Begin transaction	c.setAutoCommit(false);	Makes related updates atomic.
Lock a flight	SELECT ... WHERE flight_id=? FOR UPDATE	Serializes competing seat reservations for this flight.
Commit success	c.commit();	Persists the booking group together.
Rollback failure	c.rollback();	Removes partial writes after an error.
Cabin boundary	ordinal <= businessCapacity	First seat ordinals belong to Business.
Normalize a seat	seat.trim().toUpperCase(Locale.ROOT)	Makes 12a and 12A compare consistently.
Disable taken seat	button.setEnabled(false);	Prevents selection in the UI.
Refresh live map	new Timer(2500, e -> refreshOccupiedSeats(...))	Queries database again every 2.5 seconds.
48-hour threshold	seconds >= 48L * 3600	Selects the full-refund tier.
Round currency	divide(100, 2, HALF_UP)	Keeps money at cents precision.
Check admin passkey	if (!matchesAdminPasskey(key)) throw ...	Denies admin creation/login before routing.
Clear secret chars	Arrays.fill(secret, '\0');	Best-effort clearing of temporary credential array.
Business fare	ROUND(s.fare*2.75,2)	Sets seeded Business price to 2.75 times Economy.

Sample answer explanations

PreparedStatement: “This binds the value as data.” Transaction: “This is the atomic boundary.” FOR UPDATE: “This protects the last seat from concurrent claims.” Timer: “This refreshes the screen but does not replace checkout validation.” Passkey check: “This is a second factor for admin role, separate from the account password.”

PRACTICE RULE

When a line is deleted, restore the invariant it enforces, not just a syntactically similar line. Ask whether removing it could leak data, skip validation, corrupt inventory or permit double booking.

12.0 | Presentation checklist & glossary

A concise walkthrough should show architecture, security, a search, a seat selection, and the cancellation approval path.

TOPIC	PRESENT / EXPLAIN
Architecture	Trace a button action -> service -> DAO -> SQL -> result shown in Swing.
Security	Differentiate password hashing from encryption; explain passkey is an additional admin check.
Search	Show the optional Economy fare predicate and date range.
Concurrency	Explain display polling versus final transactional lock.
ER model	Draw user -> booking -> passenger/ticket and flight -> airline.
Refund	Compute one sample refund and identify approval plus persisted summary.
Limits	Payment/refund are simulated; DB is MySQL 8+, not SQLite.

Glossary

TERM	MEANING
JFrame	Top-level Swing window.
JPanel	Container for controls and layout.
JDialog	Modal or modeless popup window.
DAO	Data Access Object: isolates SQL persistence.
PreparedStatement	Parameterized SQL with bound values.
Transaction	Atomic set of DB work: commit all or roll all back.
Foreign key	Constraint connecting a child row to a parent row.
PBKDF2	Password-based key derivation used for salted hashes.
Row lock	Database lock that serializes booking work for a flight.
Model / DTO	Java object carrying business or result data.
MVC-style	Separation of user interface, business logic, persistence.

File extensions: .java is Java source. ui, dao, model, service, config and security are Java package names. Swing is the UI framework; there is no .swing extension.